
ОРГАНІЗАЦІЯ БАЗ ДАНИХ ТА ЗНАНЬ

Запити в PostgreSQL

Оператор **DISTINCT** дозволяє вибрати унікальні дані за певними стовпцями.

Наприклад, в таблиці товарів різні товари можуть мати одних і тих же виробників. Наприклад, у нас наступна таблиця:

```
1 CREATE TABLE Products
2 (
3     Id SERIAL PRIMARY KEY,
4     ProductName VARCHAR(30) NOT NULL,
5     Manufacturer VARCHAR(20) NOT NULL,
6     ProductCount INTEGER DEFAULT 0,
7     Price NUMERIC
8 );
9 INSERT INTO Products (ProductName, Manufacturer, ProductCount, Price)
10 VALUES
11 ('iPhone X', 'Apple', 2, 71000),
12 ('iPhone 8', 'Apple', 3, 56000),
13 ('Galaxy S9', 'Samsung', 6, 56000),
14 ('Galaxy S8 Plus', 'Samsung', 2, 46000),
15 ('Desire 12', 'HTC', 3, 26000);
```

Виберемо всіх виробників:

```
1 SELECT DISTINCT Manufacturer FROM Products;
```

```
1 select manufacturer from products;
```

Data Output Explain Messages Query History

	manufacturer character varying (20)
1	Apple
2	Apple
3	Samsung
4	Samsung
5	HTC

```
1 select distinct manufacturer from products;
```

Data Output Explain Messages Query History

	manufacturer character varying (20)
1	HTC
2	Samsung
3	Apple

Запити SQL з використанням **DISTINCT** дозволяють отримати унікальні значення з вказаного стовпця або комбінацію стовпців.

```
SELECT DISTINCT room_type FROM rooms;
```

```
room_type
```

```
-----
```

```
Single
```

```
Double
```

```
Suite
```

```
SELECT DISTINCT status FROM bookings;
```

```
status
```

```
-----
```

```
Confirmed
```

```
Pending
```

```
SELECT DISTINCT payment_method FROM payments;
```

```
payment_method
```

```
-----
```

```
Credit Card
```

```
Cash
```

```
SELECT DISTINCT address FROM clients;
```

```
address
```

```
-----
```

```
123 Main St
```

ORDER BY. Сортування.

Оператор ORDER BY дозволяє впорядкувати значення за певним стовпцем.

Наприклад, впорядкуємо вибірку з таблиці Products по стовпцю ProductCount:

```
1 select * from products
2 order by productcount;
```

Data Output	Explain	Messages	Query History		
	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	11	iPhone X	Apple	2	71000
2	14	Galaxy S8 Plus	Samsung	2	46000
3	12	iPhone 8	Apple	3	56000
4	15	Desire 12	HTC	3	26000
5	13	Galaxy S9	Samsung	6	56000

Також можна робити упорядкування даних за псевдонімом стовпчика, який визначається за допомогою оператора **AS:**

```
1 SELECT ProductName, ProductCount * Price AS TotalSum
2 FROM Products
3 ORDER BY TotalSum;
```

Data Output

[Explain](#)

[Messages](#)

[Query History](#)

	productname character varying (30)	totalsum numeric
1	Desire 12	78000
2	Galaxy S8 Plus	92000
3	iPhone X	142000
4	iPhone 8	168000
5	Galaxy S9	336000

Як критерій сортування також можна використовувати складний вираз на основі стовпців:

1
2
3

```
SELECT ProductName, Price, ProductCount
FROM Products
ORDER BY ProductCount * Price;
```

Data Output

Explain

Messages

Query History

	productname character varying (30)	price numeric	productcount integer
1	Desire 12	26000	3
2	Galaxy S8 Plus	46000	2
3	iPhone X	71000	2
4	iPhone 8	56000	3
5	Galaxy S9	56000	6

Сортування за спаданням

За замовчуванням дані сортуються за зростанням, проте за допомогою оператора **DESC** можна задати сортування за спаданням.

```
1 SELECT ProductName, Manufacturer
2 FROM Products
3 ORDER BY Manufacturer DESC;
```

Data Output			Explain	Messages	Query History
	productname character varying (30)	manufacturer character varying (20)			
1	Galaxy S9	Samsung			
2	Galaxy S8 Plus	Samsung			
3	Desire 12	HTC			
4	iPhone X	Apple			
5	iPhone 8	Apple			

За замовчуванням замість **DESC** використовується оператор **ASC** , який сортує по зростанню:

```
1 SELECT ProductName, Manufacturer
2 FROM Products
3 ORDER BY Manufacturer ASC;
```

Сотування за кількома стовпцями

Якщо необхідно впорядкувати відразу за кількома стовпцями, то всі вони перераховуються через кому після оператора **ORDER BY** :

```
1 SELECT ProductName, Price, Manufacturer
2 FROM Products
3 ORDER BY Manufacturer, ProductName;
```

За допомогою **ASC** і **DESC** можна окремо для різних стовпців визначити сортування за зростанням і за спаданням:

```
1 SELECT ProductName, Price, Manufacturer
2 FROM Products
3 ORDER BY Manufacturer ASC, ProductName DESC;
```

Data Output Explain Messages Query History

	productname character varying (30)	price numeric	manufacturer character varying (20)
1	iPhone X	71000	Apple
2	iPhone 8	56000	Apple
3	Desire 12	26000	HTC
4	Galaxy S9	56000	Samsung
5	Galaxy S8 Plus	46000	Samsung

Сортування текстових значень у SQL зазвичай відбувається в алфавітному порядку, де букви розташовані в порядку їх ASCII-кодів. Отже, літери великого регістру сортуються перед літерами малого регістру.

У зазвичай сортуванні текстових значень ASC (у зростаючому порядку) використовується алфавітний порядок (від А до Z), а DESC (у спадаючому порядку) - алфавітний порядок в зворотньому напрямку (від Z до А).

```
SELECT * FROM rooms
ORDER BY price ASC;
```

room_id	room_number	room_type	price	bed_count	bed_type	amenit
1	101	Single	75.00	1	Single	{TV,Ai
2	102	Double	100.00	2	Double	{TV,Ai
3	103	Suite	150.00	2	King-size	{TV,Ai


```
SELECT * FROM bookings
ORDER BY start_date DESC;
```

booking_id	client_id	room_id	start_date	end_date	adults	ch
2	1	2	2023-09-22	2023-09-24	2	
1	1	1	2023-09-20	2023-09-25	1	

Отримання діапазону рядків. LIMIT і OFFSET

Оператор **LIMIT** дозволяє дістати певну кількість рядків:

```
1 SELECT * FROM Products
2 ORDER BY ProductName
3 LIMIT 4;
```

Data Output Explain Messages Query History					
	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	15	Desire 12	HTC	3	26000
2	14	Galaxy S8 Plus	Samsung	2	46000
3	13	Galaxy S9	Samsung	6	56000
4	12	iPhone 8	Apple	3	56000

Оператор **OFFSET** дозволяє вказати, з якого рядка треба починати вибірку.

Наприклад, виберемо 3 рядки, починаючи з 2-го:

```
1 SELECT * FROM Products
2 ORDER BY ProductName
3 LIMIT 3 OFFSET 2;
```

Data Output						Explain	Messages	Query History
	id	productname	manufacturer	productcount	price			
	integer	character varying (30)	character varying (20)	integer	numeric			
1	13	Galaxy S9	Samsung	6	56000			
2	12	iPhone 8	Apple	3	56000			
3	11	iPhone X	Apple	2	71000			

Якщо нам треба вибрати взагалі всі рядки, починаючи з якогось конкретного, то оператор **LIMIT** можна опустити:

```
1 SELECT * FROM Products
2 ORDER BY ProductName
3 OFFSET 2;
```

Або після **LIMIT** вказати ключове слово **ALL** :

```
1 SELECT * FROM Products
2 ORDER BY ProductName
3 LIMIT ALL OFFSET 2;
```

booking_id	client_id	room_id	start_date	end_date	adults	children
1	1	1	2023-09-20	2023-09-25	1	0
2	1	2	2023-09-22	2023-09-24	2	0
3	2	3	2023-10-01	2023-10-05	2	0
4	3	2	2023-10-15	2023-10-18	1	0
5	4	1	2023-10-20	2023-10-22	2	0
6	5	4	2023-11-01	2023-11-05	2	0

```
SELECT * FROM bookings
LIMIT 5;
```

booking_id	client_id	room_id	start_date	end_date	adults	children
1	1	1	2023-09-20	2023-09-25	1	0
2	1	2	2023-09-22	2023-09-24	2	0
3	2	3	2023-10-01	2023-10-05	2	0
4	3	2	2023-10-15	2023-10-18	1	0
5	4	1	2023-10-20	2023-10-22	2	0
6	5	4	2023-11-01	2023-11-05	2	0

```
SELECT * FROM bookings
LIMIT 5;
```

Запит на вибірку перших 5 бронювань з бази даних:

booking_id	client_id	room_id	start_date	end_date	adults	children
1	1	1	2023-09-20	2023-09-25	1	0
2	1	2	2023-09-22	2023-09-24	2	0
3	2	3	2023-10-01	2023-10-05	2	0
4	3	2	2023-10-15	2023-10-18	1	0
5	4	1	2023-10-20	2023-10-22	2	0

booking_id	client_id	room_id	start_date	end_date	adults	children
1	1	1	2023-09-20	2023-09-25	1	0
2	1	2	2023-09-22	2023-09-24	2	0
3	2	3	2023-10-01	2023-10-05	2	0
4	3	2	2023-10-15	2023-10-18	1	0
5	4	1	2023-10-20	2023-10-22	2	0
6	5	4	2023-11-01	2023-11-05	2	0

```
SELECT * FROM bookings
OFFSET 5
LIMIT 5;
```

booking_id	client_id	room_id	start_date	end_date	adults	child
1	1	1	2023-09-20	2023-09-25	1	
2	1	2	2023-09-22	2023-09-24	2	
3	2	3	2023-10-01	2023-10-05	2	
4	3	2	2023-10-15	2023-10-18	1	
5	4	1	2023-10-20	2023-10-22	2	
6	5	4	2023-11-01	2023-11-05	2	

Запит на вибірку наступних 5 бронювань після перших 5:

```
SELECT * FROM bookings
OFFSET 5
LIMIT 5;
```

Результат (рядки з 6 по 10):

booking_id	client_id	room_id	start_date	end_date	adults	child
6	5	4	2023-11-01	2023-11-05	2	

booking_id	client_id	room_id	start_date	end_date	adults	children
1	1	1	2023-09-20	2023-09-25	1	0
2	1	2	2023-09-22	2023-09-24	2	0
3	2	3	2023-10-01	2023-10-05	2	0
4	3	2	2023-10-15	2023-10-18	1	0
5	4	1	2023-10-20	2023-10-22	2	0
6	5	4	2023-11-01	2023-11-05	2	0

```
SELECT * FROM bookings
OFFSET 3
LIMIT 3;
```

booking_id	client_id	room_id	start_date	end_date	adults	children
1	1	1	2023-09-20	2023-09-25	1	0
2	1	2	2023-09-22	2023-09-24	2	0
3	2	3	2023-10-01	2023-10-05	2	0
4	3	2	2023-10-15	2023-10-18	1	0
5	4	1	2023-10-20	2023-10-22	2	0
6	5	4	2023-11-01	2023-11-05	2	0

Запит на вибірку наступних 3 бронювань після перших 3:

```
SELECT * FROM bookings
OFFSET 3
LIMIT 3;
```

Результат (рядки з 4 по 6):

booking_id	client_id	room_id	start_date	end_date	adults	children
4	3	2	2023-10-15	2023-10-18	1	0
5	4	1	2023-10-20	2023-10-22	2	0
6	5	4	2023-11-01	2023-11-05	2	0

Оператори фільтрації

Оператор IN

Оператор **IN** дозволяє визначити набір значень, які повинні мати стовпці:

```
1 | WHERE вираження [NOT] IN (вираження)
```

Вираз в дужках після **IN** визначає набір значень. Цей набір може обчислюватися динамічно на підставі, наприклад, ще одного запиту, або це можуть бути константні значення.

Наприклад, виберемо товари, у яких виробник або Samsung, або Xiaomi, або Huawei:

```
1 SELECT * FROM Products
2 WHERE Manufacturer IN ('Samsung', 'HTC', 'Huawei');
```

Data Output		Explain	Messages	Query History	
	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	13	Galaxy S9	Samsung	6	56000
2	14	Galaxy S8 Plus	Samsung	2	46000
3	15	Desire 12	HTC	3	26000

Як альтернативою можна було б перевірити всі ці значення через оператор **OR**:

```
1 SELECT * FROM Products
2 WHERE Manufacturer = 'Samsung' OR Manufacturer = 'HTC' OR Manufacturer = 'Huawei';
```

Однак використання оператора IN набагато зручніше, особливо якщо подібних значень дуже багато.

За допомогою оператора **NOT** можна знайти всі рядки, які, навпаки, не відповідають набору значень:

```
1 SELECT * FROM Products
2 WHERE Manufacturer NOT IN ('Samsung', 'HTC', 'Huawei');
```

Оператор **IN** в SQL використовується для фільтрації рядків на основі наявності значення стовпця у певному списку значень. Він дозволяє вибрати рядки, в яких значення вказаного стовпця збігаються з будь-яким зі значень, перелічених у списку.

Ось загальний синтаксис використання **IN**:

```
SELECT * FROM table_name
WHERE column_name IN (value1, value2, ...);
```

- `table_name`: Назва таблиці, з якої ви хочете вибрати дані.
- `column_name`: Назва стовпця, значення якого ви хочете порівняти зі списком значень.
- `value1, value2, ...`: Значення, з якими ви хочете порівняти значення стовпця. Вони перераховуються у вигляді списку значень у круглих дужках і можуть бути конкретними значеннями, виразами чи підзапитами.

Оператор **IN** повертає **TRUE**, якщо значення стовпця дорівнює одному зі значень у списку. В іншому випадку, він повертає **FALSE**.

```
-- Виберемо всіх клієнтів, чий ідентифікатор дорівнює 1 або 3 або 5
SELECT * FROM clients
WHERE client_id IN (1, 3, 5);

-- Виберемо всі товари, які є або "Літня сукня" або "Сонцезахисні окуляри"
SELECT * FROM products
WHERE product_name IN ('Літня сукня', 'Сонцезахисні окуляри');
```

Такий оператор дозволяє вам зручно фільтрувати рядки за допомогою списку значень та зменшує необхідність в багаторазових умовних операціях **OR**.

Запити SQL з використанням операторів **IN** та **NOT IN** використовуються для фільтрації даних на основі наявності або відсутності значень у певному списку.

```
-- Виберемо бронювання для клієнтів, ідентифікатори яких не входять до списку
SELECT * FROM bookings
WHERE client_id NOT IN (2, 4);
```

booking_id	client_id	room_id	start_date	end_date	adults	children
1	1	1	2023-09-20	2023-09-25	1	0
4	3	2	2023-10-15	2023-10-18	1	0
6	5	4	2023-11-01	2023-11-05	2	0

Оператор BETWEEN

Оператор BETWEEN визначає діапазон значень за допомогою початкового і кінцевого значення, яким має відповідати вираз:

```
1 WHERE выражение [NOT] BETWEEN начальное_значение AND конечное_значение
```

Наприклад, отримаємо усі товари, у яких ціна від 20 000 до 50 000 (початкове і кінцеве значення також включаються в діапазон):

```
1 SELECT * FROM Products
2 WHERE Price BETWEEN 20000 AND 50000;
```

Data Output						Explain	Messages	Query History
	id	productname	manufacturer	productcount	price			
	integer	character varying (30)	character varying (20)	integer	numeric			
1	14	Galaxy S8 Plus	Samsung	2	46000			
2	15	Desire 12	HTC	3	26000			

Якщо треба, навпаки, вибрати ті рядки, які не потрапляють в даний діапазон, то застосовується оператор **NOT**:

```
1 SELECT * FROM Products
2 WHERE Price NOT BETWEEN 20000 AND 50000;
```

Також можна використовувати більш складні вирази. Наприклад, отримаємо товари, запаси яких на певну суму (ціна * кількість):

```
1 SELECT * FROM Products
2 WHERE Price * ProductCount BETWEEN 90000 AND 150000;
```

Оператор LIKE

Оператор LIKE приймає шаблон рядка, якому має відповідати вираз.

```
1 WHERE выражение [NOT] LIKE шаблон_строки
```

```
1 SELECT * FROM Products
2 WHERE ProductName LIKE 'iPhone%';
```

Data Output Explain Messages Query History					
	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	11	iPhone X	Apple	2	71000
2	12	iPhone 8	Apple	3	56000

Для визначення шаблону можуть застосовуватися ряд спеціальних символів підстановки:

% : Відповідає будь-якому підрядку, який може мати будь-яку кількість символів, при цьому підрядок може і не містити жодного символу. Наприклад, вираз `WHERE ProductName LIKE 'Galaxy%'` відповідає таким значенням як "Galaxy Ace 2" або "Galaxy S7"

_ : Відповідає будь-якому одиночному символу. Наприклад, вираз `WHERE ProductName LIKE 'Galaxy S_'` відповідає таким значенням як "Galaxy S7" або "Galaxy S8".

EXIST

PostgreSQL умова EXISTS використовується у поєднанні з підзапитом і вважається "виконаним", якщо підзапит повертає хоча б один рядок. Його можна використовувати в операторі [SELECT](#) , [INSERT](#) , [UPDATE](#) або [DELETE](#) .

Синтаксис

```
WHERE EXISTS (subquery);
```


Оператор EXISTS в SQL використовується для перевірки наявності результатів підзапиту. Він повертає TRUE, якщо підзапит повертає хоча б один рядок результату, і FALSE, якщо підзапит не повертає жодного рядка результату.

Ось загальний синтаксис оператора EXISTS:

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS (subquery);
```

- `column_name(s)`: Список стовпців, які ви хочете вибрати у головному запиті.
- `table_name`: Назва таблиці, з якої ви хочете вибрати дані у головному запиті.
- `subquery`: Підзапит, результат якого перевіряється за допомогою оператора EXISTS.

Призначення оператора EXISTS полягає в тому, щоб виконати головний запит тільки в тому випадку, коли підзапит має хоча б один результат. Це дозволяє вам створювати більш складні запити, що базуються на наявності даних у підзапиті.

Ось приклади використання оператора EXISTS:

Знайти всіх клієнтів, які мають бронювання:

```
SELECT client_id, client_name
FROM clients
WHERE EXISTS (SELECT 1 FROM bookings WHERE bookings.client_id = clients.client_id)
```

Знайти всі продукти, які мають хоча б один відгук:

```
SELECT product_id, product_name  
FROM products  
WHERE EXISTS (SELECT 1 FROM reviews WHERE reviews.product_id = products.produ
```

Знайти всі країни, у яких є принаймні одне місто в базі даних:

```
SELECT DISTINCT country  
FROM locations  
WHERE EXISTS (SELECT 1 FROM cities WHERE cities.country = locations.country)
```

Приклад – з оператором SELECT

```
SELECT *  
FROM products  
WHERE EXISTS (SELECT 1  
               FROM inventory  
               WHERE products.product_id = inventory.product_id);
```

У цьому прикладі PostgreSQL умови EXISTS будуть повернуті всі записи з таблиці `products`, де в таблиці `inventory` є хоча б один запис з співпадаючим `product_id`. Ми використовували `SELECT 1` у підзапиті для підвищення продуктивності, оскільки результуючий набір стовпців не має відношення до умови EXISTS (має значення лише наявність поверненого рядка).

Приклад - оператор SELECT з використанням NOT EXISTS

```
SELECT *  
FROM products  
WHERE NOT EXISTS (SELECT 1  
                  FROM inventory  
                  WHERE products.product_id = inventory.product_id);
```

Приклад – з оператором INSERT

```
INSERT INTO contacts  
(contact_id, contact_name)  
SELECT supplier_id, supplier_name  
FROM suppliers  
WHERE EXISTS (SELECT 1  
              FROM orders  
              WHERE suppliers.supplier_id = orders.supplier_id);
```

Приклад – з оператором UPDATE

```
UPDATE suppliers
SET supplier_name = (SELECT customers.customer_name
                     FROM customers
                     WHERE customers.customer_id = suppliers.supplier_id)
WHERE EXISTS (SELECT 1
              FROM customers
              WHERE customers.customer_id = suppliers.supplier_id);
```

Приклад – з оператором DELETE

```
DELETE FROM contacts
WHERE EXISTS (SELECT 1
              FROM employees
              WHERE contacts.contact_id = employees.employee_id);
```


ANY, SOME, ALL

Якщо підзапиту **передуватиме ключове слово ALL**, умова порівняння вважається виконаною тільки в тому випадку, якщо вона **виконується для всіх значень** в результуючому стовпці підзапиту.

Якщо підзапиту **передує ключове слово ANY**, то умова порівняння буде вважатися виконаною, якщо вона **задовольняється хоча б для будь-якого** (одного або декількох) **значень** в результуючому стовпці підзапиту.

Відповідно до стандарту ISO додатково можна використовувати ключове слово SOME, що є синонімом ключового слова ANY.

Оператори **ANY**, **SOME**, і **ALL** використовуються для порівняння значень стовпців зі значеннями, які повертає підзапит. Ось приклади використання цих операторів:

ANY або **SOME** з оператором порівняння

Приклад використання **ANY** (або **SOME**) з оператором порівняння для порівняння значень зі списком у підзапиті:

```
-- Знайдемо всі продукти, які коштують принаймні стільки, скільки коштує продукт з ID=1
SELECT product_id, product_name
FROM products
WHERE product_price >= ANY (SELECT product_price FROM products WHERE product_id = 1);
```

У цьому прикладі ми порівнюємо ціни інших продуктів з ціною продукту з ID=1. Якщо ціна іншого продукту більше або дорівнює ціні продукту з ID=1, вона включається до результату.

What's the difference between ANY and SOME?

Оператори ANY і SOME використовуються в SQL для порівняння значень зі списком, що повертається підзапитом. Фактично, ці два оператори є взаємозамінними, і різниця між ними полягає лише в стилі написання запиту.

- ANY: Використовується для порівняння значення зі списком і повертає TRUE, якщо хоча б одне значення у списку задовольняє умову.
- SOME: Це синонім для ANY. Ви можете використовувати SOME так само, як і ANY, і він працює точно так само.

-- Виберемо всіх клієнтів, які мають бронювання

```
SELECT client_id, client_name  
FROM clients c  
WHERE client_id = ANY (SELECT client_id FROM bookings);
```

Цей запит вибере всіх клієнтів, які мають бронювання, використовуючи оператор **ANY**. 51

Але ви можете написати аналогічний запит, використовуючи **SOME**, і результат буде той самий:

```
-- Виберемо всіх клієнтів, які мають бронювання (використовуючи SOME)
SELECT client_id, client_name
FROM clients c
WHERE client_id = SOME (SELECT client_id FROM bookings);
```

Основна різниця між ними - це стиль написання запиту. Вибір між **ANY** і **SOME** залежить від вашого особистого стилю програмування та вашого вибору. Вони обидва функціонують ідентично.

SQL запит з використанням оператора WHERE ALL дозволяє фільтрувати рядки в результатах запиту на основі умови, яку повинні виконувати всі записи в підзапиті. Ось приклад такого запиту:

```
SELECT *
FROM Orders
WHERE ALL (
    SELECT 1
    FROM OrderDetails
    WHERE OrderDetails.OrderID = Orders.OrderID
        AND OrderDetails.Price > 50
);
```

Припустимо, у нас є дві таблиці: "Замовлення" (Orders) та "Деталі замовлення" (OrderDetails). Ми хочемо вибрати всі замовлення, в яких всі деталі мають ціну більше 50.

```
SELECT *  
FROM Orders  
WHERE ALL (  
    SELECT 1  
    FROM OrderDetails  
    WHERE OrderDetails.OrderID = Orders.OrderID  
        AND OrderDetails.Price > 50  
);
```

У цьому прикладі головний запит вибирає всі рядки з таблиці "Замовлення", де всі записи в підзапиті виконують умову, що "ціна" (Price) у таблиці "Деталі замовлення" (OrderDetails) більше 50 для кожного відповідного "OrderID". Якщо ця умова виконується для всіх записів в підзапиті, то відповідний запис у таблиці "Замовлення" включається до результату запиту.

Зверніть увагу, що `SELECT 1` в підзапиті використовується лише для перевірки умови і не повертає реальних даних. Ви можете вибрати конкретні поля з таблиці "Замовлення", які вам потрібні, замість вибору всіх (*) в зовнішньому запиті.